

PROJECT IMPLEMENTATION GUIDE

S-MAUMe

Hybrid RAG 기반 대학 캠퍼스 AI Agent

MVP 전체 폴더 구조 및 단계별 개발 순서

기준 문서: PRD · 최종 제작설계서 · 디자인 명세서 · 개발보고서 · 수행계획서 · 논문

작성 기준일 2026. 09. 03 | 구현 전 계획 문서

1. 문서 목적과 적용 원칙

이 문서는 S-MAUMe 프로젝트를 처음 구현하는 개발자가 전체 구조를 이해하고, 작은 단위로 실행 결과를 확인하면서 개발하기 위한 기준 가이드다. 실제 코드를 한꺼번에 생성하지 않고, 각 단계의 통과 조건을 만족한 뒤 다음 단계로 진행한다.

핵심 원칙 FAQ 직접 응답 → Vector RAG → LangGraph 라우팅 → Neo4j 경로 탐색 → 복합 Hybrid RAG 순서로 확장한다.

기준 기술 스택

1. Frontend: React
2. Backend: FastAPI
3. AI Agent: LangChain + LangGraph
4. LLM 및 임베딩: OpenAI API
5. RDB: PostgreSQL
6. Vector DB: Qdrant
7. Graph DB: Neo4j
8. 운영 환경: AWS Ubuntu
9. 컨테이너 실행: Docker Compose

개발 진행 규칙

10. 기준 문서에 없는 기능을 임의로 추가하지 않는다.
11. 한 번에 기능 하나 또는 데이터 경로 하나만 구현한다.
12. 각 단계는 실행·테스트·결과 확인을 통과해야 완료된다.
13. 실제 비밀키와 개인정보는 코드 및 저장소에 남기지 않는다.
14. Docker Compose 환경에서도 같은 결과가 나오는지 조기에 확인한다.

2. MVP 기능 범위

PRD 의 P0 우선순위를 MVP 기준으로 사용한다. 디자인 명세서에만 존재하거나 PRD 에서 P1-P2 로 분류된 기능은 기본적으로 후속 단계로 분리한다.

영역	MVP 포함	핵심 완료 기준
자연어 Q&A	학사 일정·공지·학칙 RAG 답변	교내 근거와 출처를 함께 반환
FAQ	전화번호·운영시간 등 직접 응답	LLM 을 호출하지 않고 PostgreSQL 에서 응답
예외 처리	근거 부족·검색 실패 안내	임의 답변 없이 추가 질문을 유도
캠퍼스 안내	건물·강의실·시설 및 연결 경로	Neo4j 검색 결과를 지도/위치 UI 에 표시
사용자 화면	In-Page 채팅, 쿼리메뉴, 답변 카드	질문부터 출처 확인까지 한 화면에서 완료
데이터 관리	PDF 업로드, FAQ·시설 수기 등록	등록 데이터가 검색에 반영되고 재시작 후 유지
보안·로그	개인정보 마스킹, 최소 익명 로그	이름·학번·연락처 원문이 저장되지 않음
운영	Docker Compose, AWS Ubuntu, HTTPS	새 서버에서 문서화된 절차로 재현 가능

MVP 이후로 분리할 기능

15. 선수과목·조직도 전체 그래프와 고급 관계 시각화
16. 날씨 API 를 활용한 상황 인식 및 실내 경로 우선 추천
17. 학과별 개인화와 스펙 관리
18. 자동완성·연관 질문 추천 고도화
19. 다국어 전체 지원, 상담사 실시간 연결, 완성형 다크 모드
20. 통학버스와 학식 정보의 고도화 기능

3. 구현 전 반드시 확정할 설계 항목

기준 문서 사이에 일부 표현 차이가 있다. 아래 항목은 폴더 구조와 무관하게 수용할 수 있지만, 실제 UI 또는 서비스 통신을 구현하기 전에 한 가지 기준으로 확정해야 한다.

항목	문서에 나타난 선택지	구현 전 조치
챗봇 형태	In-Page 슬라이드 패널 / 700x580 팝업 위젯	최종 동작 방식 1 개 확정
브랜드 컬러	딥 로즈 #8B1A4A / 퍼플 #635B97	디자인 토큰 기준 확정
퀵메뉴	PRD 4 종 / 디자인 명세서 6 종	MVP 노출 항목 확정
부가 기능	다국어·다크모드·상담 연결·만족도	MVP 포함 여부 확정
내부 통신	Backend 와 AI Agent 컨테이너 분리	HTTP 등 통신 방식 확정

중요 위 항목을 확정하기 전에는 임의로 한쪽 문서를 선택해 UI 나 통신 구조를 구현하지 않는다.

4. 권장 프로젝트 구조

문서에 명시된 Frontend, Backend, AI Engine 분리 원칙을 반영한다. 하나의 저장소 안에서 세 실행 단위를 나누고, Docker Compose 가 이를 PostgreSQL·Qdrant·Neo4j 와 함께 실행하도록 구성한다.

```
D:\projects\s-maume\
├─ docs/                기준 문서
├─ apps/
│   └─ frontend/       React 사용자·관리자 화면
│       ├── public/
│       └─ src/
│           ├── app/    앱 시작점·라우팅·전역 설정
│           ├── pages/
│           │   ├── user/
│           │   └─ admin/
│           └─ features/
│               ├── chat/
│               ├── quick-menu/
│               ├── campus-map/
│               ├── academic/
│               ├── cafeteria/
│               ├── extracurricular/
│               ├── departments/
│               └─ data-management/
│   ├── components/
│   ├── design-system/
│   ├── services/
│   ├── hooks/
│   ├── types/
│   └─ assets/
```

├─ tests/	
├─ Dockerfile	
└─ backend/	외부 공개 FastAPI 서버
├─ app/	
├─ main.py	
├─ api/routes/	health·chat·feedback·admin
├─ core/	설정·로그·보안
├─ middleware/	개인정보 마스크
├─ schemas/	API 요청·응답 형식
├─ models/	PostgreSQL 모델
├─ repositories/	PostgreSQL 접근
├─ services/	업무 처리
├─ clients/	AI Agent 호출
└─ db/postgres.py	
├─ migrations/	
├─ tests/	
└─ Dockerfile	
└─ services/	
├─ ai-agent/	LangChain + LangGraph
├─ app/	
├─ interface/	Backend 연결 인터페이스
├─ graph/	state·workflow·nodes
├─ retrievers/	FAQ·RDB·Vector·Graph
├─ ingestion/	입력·정제·분할·색인
├─ prompts/	
├─ schemas/	
├─ clients/	OpenAI·각 데이터베이스
└─ core/	
├─ tests/	
└─ Dockerfile	
└─ data/	
├─ samples/	비민감 테스트 데이터
├─ seeds/	FAQ·시설 초기 데이터
└─ schemas/	데이터 입력 형식
└─ infra/	
├─ postgres/	
├─ qdrant/	
├─ neo4j/	
└─ aws/	
└─ scripts/	
├─ development/	
├─ ingestion/	
└─ operations/	
└─ tests/e2e/	
└─ docker-compose.yml	
└─ docker-compose.dev.yml	
└─ .env.example	
└─ .gitignore	
└─ README.md	

폴더 생성 원칙 위 트리는 최종 예상 구조다. 시작할 때 전부 만들지 말고, 해당 단계에서 실제로 사용하는 폴더만 추가한다.

구성 요소별 책임

21.Frontend: 화면 표시와 사용자 입력만 담당하며 DB 에 직접 연결하지 않는다.

22.Backend: 공개 API, 인증·보안, 개인정보 마스크, 로그 및 관리자 요청을 담당한다.

23.**AI Agent**: 질문 분류, 검색 계획, DB 별 검색, 답변 생성·평가·예외 처리를 담당한다.

24.**PostgreSQL**: FAQ, 시설, 메뉴, 시간표, 익명 로그 등 정형 데이터를 저장한다.

25.**Qdrant**: 학사 공지·학칙·장학금·비교과 문서의 임베딩과 메타데이터를 저장한다.

26.**Neo4j**: 건물 연결, 강의실 위치, 선수과목, 부서 관계를 노드와 관계로 저장한다.

5. 단계별 개발 순서

아래 순서는 문서의 분석→설계→개발→테스트→배포 흐름을 유지하면서, 초보자가 오류 원인을 좁힐 수 있도록 하나의 실행 경로씩 추가하는 방식으로 세분화한 것이다.

단계 0 — 설계 기준 고정

목표: 문서 간 표현 차이를 해결하고 MVP 경계를 잠근다.

작업

- 27. 챗봇 표시 방식, 브랜드 컬러, 퀵메뉴 항목을 확정한다.
- 28. MVP 와 후속 기능 목록을 분리한다.
- 29. Backend-AI Agent 내부 통신 방식을 확정한다.

통과 조건

- 30. 미결정 항목이 체크리스트에 남아 있지 않다.
- 31. 모든 팀원이 동일한 MVP 목록을 사용한다.

단계 1 — FastAPI 최소 서버

목표: 가장 작은 백엔드 프로그램이 로컬에서 실행되게 한다.

작업

- 32. FastAPI 애플리케이션을 실행한다.
- 33. 환경설정 로딩과 /health API 만 만든다.
- 34. 상태 확인 테스트 하나를 작성한다.

통과 조건

- 35. 브라우저 또는 API 문서에서 정상 응답을 확인한다.
- 36. 테스트를 다시 실행해도 같은 결과가 나온다.

단계 2 — React 최소 화면

목표: 브라우저에서 Backend 연결 여부를 확인한다.

작업

- 37.React 애플리케이션을 실행한다.
- 38.FastAPI /health 를 호출한다.
- 39.연결 성공·실패 상태를 화면에 표시한다.

통과 조건

- 40.React 화면에 Backend 연결 성공이 표시된다.
- 41.Backend 를 중지하면 오류 상태가 표시된다.

단계 3 — AI Agent 최소 연결

목표: 실제 AI 기능 없이 세 실행 단위의 왕복 경로를 검증한다.

작업

- 42.AI Agent 내부 인터페이스를 만든다.
- 43.입력 문장을 그대로 반환하는 테스트 응답을 만든다.
- 44.Backend 가 AI Agent 를 호출하도록 연결한다.

통과 조건

- 45.React → FastAPI → AI Agent → FastAPI → React 왕복이 성공한다.
- 46.OpenAI API 와 데이터베이스는 아직 사용하지 않는다.

단계 4 — Docker 기본 실행

목표: Frontend·Backend·AI Agent 를 동일한 명령으로 실행한다.

작업

- 47. 세 실행 단위의 Dockerfile 을 만든다.
- 48. Docker Compose 서비스와 내부 네트워크를 정의한다.
- 49. 환경변수는 .env 에서 주입하고 비밀값은 저장소에 넣지 않는다.

통과 조건

- 50. docker compose up 으로 세 서비스가 기동한다.
- 51.3 단계의 왕복 테스트가 컨테이너 환경에서도 성공한다.

중간 저장점 여기까지 안정적으로 실행되는 상태를 Git 커밋으로 남긴 뒤 다음 단계로 진행한다.

단계 5 — PostgreSQL 과 FAQ 직접 응답

목표: LLM 없이 처리되는 가장 단순한 실제 기능을 완성한다.

작업

- 52. PostgreSQL 컨테이너와 마이그레이션을 추가한다.
- 53. FAQ 테이블과 테스트 데이터 5~10 개를 만든다.
- 54. 키워드 기반 FAQ 조회와 응답을 연결한다.

통과 조건

- 55. 전화번호·운영시간 질문이 PostgreSQL 에서 반환된다.
- 56. FAQ 응답에서 OpenAI API 가 호출되지 않는다.

단계 6 — 개인정보 마스킹과 최소 로그

목표: 사용자 입력 저장 전에 개인정보를 제거한다.

작업

- 57. 이름·학번·전화번호 마스킹 규칙을 만든다.

58. 익명 세션 ID, 질문 유형, 응답시간, LLM 호출 여부만 기록한다.

59. 마스킹 단위 테스트를 추가한다.

통과 조건

60. 개인정보가 포함된 질문 원문이 DB에 남지 않는다.

61. 필요 최소 로그만 조회된다.

단계 7 — Qdrant 문서 검색

목표: 문서 1~2 개로 색인과 검색만 검증한다.

작업

62. PDF 입력 → 텍스트 추출 → 정제 → 분할 흐름을 만든다.

63. OpenAI 임베딩을 생성해 Qdrant에 저장한다.

64. 문서 ID·제목·페이지·유효기간 메타데이터를 함께 저장한다.

65. 대표 질문으로 유사도 검색을 실행한다.

통과 조건

66. 대표 질문 10 개에서 관련 문서 조각과 출처가 검색된다.

67. 같은 문서를 재색인해도 불필요한 중복이 생기지 않는다.

단계 8 — 근거 기반 RAG 답변

목표: 검색된 교내 문서만 근거로 자연어 답변을 생성한다.

작업

68. Qdrant 검색 결과를 OpenAI API에 전달한다.

69. 근거 범위 밖의 답변을 금지하는 프롬프트를 적용한다.

70. 답변과 함께 문서 출처를 반환한다.

71. 검색 결과 없음 예외 처리를 추가한다.

통과 조건

72. 모든 RAG 답변에 확인 가능한 출처가 있다.

73. 근거가 없으면 추측하지 않고 정보 부족을 안내한다.

74. FAQ 질문은 계속 LLM 을 호출하지 않는다.

중간 저장점 여기까지 안정적으로 실행되는 상태를 Git 커밋으로 남긴 뒤 다음 단계로 진행한다.

단계 9 — LangGraph 라우팅

목표: FAQ 와 문서 질문을 자동으로 다른 경로에 보낸다.

작업

75. 질문 전처리, 의도 분류, 검색 계획, 검색, 답변, fallback 노드를 만든다.

76. 첫 버전에서는 PostgreSQL 과 Qdrant 두 경로만 구분한다.

77. 선택된 경로를 익명 로그에 기록한다.

통과 조건

78. 준비한 질문 세트가 의도한 데이터베이스로 이동한다.

79. 잘못 분류된 질문을 재현할 수 있는 테스트가 있다.

단계 10 — Neo4j 캠퍼스 경로

목표: 작은 샘플 그래프로 관계 검색을 검증한다.

작업

80. 건물 2 개, 연결 통로 1 개, 강의실과 시설 몇 개만 모델링한다.

81. 출발지-목적지 기반 경로 탐색을 구현한다.

82. Neo4j 검색 경로를 LangGraph 에 추가한다.

통과 조건

- 83.샘플 출발지와 목적지 사이의 올바른 연결 경로가 반환된다.
- 84.존재하지 않는 장소는 예외 메시지를 반환한다.

단계 11 — 캠퍼스 지도 화면

목표: 그래프 검색 결과를 사용자가 이해할 수 있게 표시한다.

작업

- 85.건물과 시설 위치를 표시한다.
- 86.선택된 목적지와 이동 경로를 강조한다.
- 87.텍스트 위치 안내를 지도와 함께 제공한다.

통과 조건

- 88.같은 경로가 API 결과와 화면에서 일치한다.
- 89.작은 화면에서도 건물명과 경로를 확인할 수 있다.

단계 12 — 복합 Hybrid RAG

목표: 필요한 질문에서만 여러 저장소를 조합한다.

작업

- 90.복수 데이터베이스 검색 계획을 만든다.
- 91.검색 결과를 공통 형식으로 정규화하고 통합·재정렬한다.
- 92.답변과 근거의 일치 여부를 평가한다.
- 93.평가 실패 시 제한된 재검색 또는 정보 부족 안내로 종료한다.

통과 조건

- 94.단일 DB 질문은 불필요한 다른 DB 를 조회하지 않는다.

95.복합 질문은 사용한 모든 출처를 구분해 보여준다.

96.재검색 루프에는 최대 횟수 제한이 있다.

중간 저장점 여기까지 안정적으로 실행되는 상태를 Git 커밋으로 남긴 뒤 다음 단계로 진행한다.

단계 13 — P0 사용자 UI 완성

목표: PRD 의 핵심 사용 흐름을 실제 화면에서 마무리한다.

작업

97.채팅 버블, 로딩 상태, 답변 카드, 출처, 오류 상태를 구현한다.

98.쿼메뉴와 대화 초기화를 연결한다.

99.캠퍼스 지도와 RAG 답변을 같은 인터페이스에서 이용하게 한다.

100. 디자인 명세서의 반응형 기준을 적용한다.

통과 조건

101. 사용자가 질문 입력부터 출처 확인까지 중단 없이 수행한다.

102. 최소 320px 과 기본 데스크톱 크기에서 핵심 기능이 동작한다.

단계 14 — 관리자 데이터 등록

목표: 운영자가 코드 수정 없이 데이터를 갱신하게 한다.

작업

103. PDF 업로드와 FAQ·시설 수기 등록 기능을 만든다.

104. 업로드 이후 전처리·분할·임베딩·색인을 연결한다.

105. 유효기간, 종료 상태, 학칙 버전을 관리한다.

통과 조건

106. 새 데이터가 검색 결과에 반영된다.

- 107. 컨테이너 재시작 후 데이터가 유지된다.
- 108. 만료 데이터는 일반 검색에서 제외되거나 종료 상태로 표시된다.

단계 15 — 통합 테스트와 KPI

목표: 기능의 정확성·속도·비용을 숫자로 확인한다.

작업

- 109. FAQ·Vector·Graph·복합 질문의 정답 세트를 만든다.
- 110. 마스킹, 출처 없음, DB 장애 상황을 테스트한다.
- 111. 응답시간과 LLM 미호출 비율을 측정한다.
- 112. 사용자 만족도 수집 흐름을 연결한다.

통과 조건

- 113. 답변 만족도 목표 80% 이상을 측정할 수 있다.
- 114. 평균 응답시간 3 초 이하 여부를 확인한다.
- 115. LLM 비용 절감률 70% 이상 여부를 확인한다.

단계 16 — AWS Ubuntu 배포

목표: 새 서버에서 동일한 서비스를 재현한다.

작업

- 116. AWS Ubuntu 에 Docker 와 Docker Compose 실행 환경을 준비한다.
- 117. 운영 환경변수와 API 키를 안전하게 주입한다.
- 118. HTTPS, 영속 볼륨, 로그, 백업·복구 절차를 구성한다.

통과 조건

- 119. 문서화된 명령만으로 전체 서비스가 기동한다.

120. 재시작 후 PostgreSQL·Qdrant·Neo4j 데이터가 유지된다.

121. 외부에서 HTTPS 로 상태 확인과 MVP 시나리오가 동작한다.

6. 단계 공통 완료 체크리스트

- 122. 실행 명령과 기대 결과가 README 에 기록되어 있다.
- 123. 정상 동작뿐 아니라 최소 한 가지 실패 상황도 확인했다.
- 124. 테스트를 다시 실행해도 동일한 결과가 나온다.
- 125. Docker Compose 환경에서도 기능이 동작한다.
- 126. 비밀번호, 개인정보, 실제 운영 데이터가 저장소에 포함되지 않았다.
- 127. 기준 문서와 구현 결과가 일치하는지 확인했다.
- 128. 다음 단계에서 사용할 인터페이스가 명확하다.

7. MVP 완료 정의

- 129. 사용자가 자연어로 FAQ·문서·위치 질문을 입력할 수 있다.
- 130. LangGraph 가 질문 유형에 맞는 데이터 경로를 선택한다.
- 131. FAQ 는 PostgreSQL 에서 LLM 없이 응답한다.
- 132. 문서 질문은 Qdrant 검색 근거와 출처를 포함한다.
- 133. 위치 질문은 Neo4j 관계 검색 결과와 지도 정보를 제공한다.
- 134. 근거 부족 시 임의 답변을 하지 않는다.
- 135. 개인정보가 마스킹된 후 최소 로그만 저장된다.
- 136. 관리자가 PDF·FAQ·시설 정보를 갱신할 수 있다.
- 137. 전체 서비스가 Docker Compose 로 실행되고 AWS Ubuntu 에서 재현된다.

다음 행동 첫 구현 작업은 전체 폴더 생성을 한꺼번에 하는 것이 아니라, 단계 0 을 확인한 뒤 단계 1 의 FastAPI /health 하나만 만드는 것이다.

8. 기준 문서

- 138. **[S-MAUMe] PRD.pdf**: MVP 우선순위, 데이터 매핑, 사용자 흐름, 보안, KPI
- 139. **최종_제작설계서.pptx**: 서비스 구성, 기능 목록, 알고리즘, 관리자 기능
- 140. **S-MAUMe_디자인명세서.pdf**: 컴포넌트, 화면, 반응형, 접근성, 상호작용
- 141. **4 차 중간보고서 개발보고서.pdf**: 최신 개발 방향, 컨테이너 구조, Agentic RAG
- 142. **한이음 드림업 프로젝트 [S-MAUMe] 수행계획서 (S4).pdf**: 사업 목표, 역할, 일정, 예상 결과물
- 143. **논문_최종.pdf**: Hybrid 지식베이스, Agentic RAG 처리·평가 구조

문서 위치: D:\projects\s-maume\docs